OpenZeppelin | security

OpenZeppelin Stack for Zama: Smart Accounts Audit (Phase Two)



April 10, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

Scope 4

System Overview 5

Privacy Guarantees 6

Security Model and Trust Assumptions 7

Critical Severity 9

C-01 Global Disclosed Mapping Shared Across Accounts Enables Cross-Account Validation Bypass 9

High Severity 10

H-01 allowHandle Does not Bind handle to account, Enabling Cross-account Permanent Decryption Grants 10

H-02 Publicly Decryptable valid Bit Acts as Oracle Over Secret Signer Set/Threshold/Weights 11

Medium Severity 12

M-01 Stored Disclosure Snapshots Remain Usable After Signer and Threshold Changes 12

Low Severity 14

L-01 allowHandle Reverts for ERC-5267 Domains That Signal Extensions (Fields Bit 0x20) 14

Notes & Additional Information 15

N-01 _discloseOperation is Externally Reachable, Weakening Wrapper-based Extension Safety 15

Conclusion 16

Appendix 17

Issue Classification 17

Summary

Type	Library
Timeline	From 2026-03-13 To 2026-03-25
Languages	Solidity

Total Issues	6 (5 resolved, 1 partially resolved)
Critical Severity Issues	1 (1 resolved)
High Severity Issues	2 (1 resolved, 1 partially resolved)
Medium Severity Issues	1 (1 resolved)
Low Severity Issues	1 (1 resolved)
Notes & Additional Information	1 (1 resolved)
Client Reported Issues	0 (0 resolved)

Scope

Scope

OpenZeppelin audited the open-accounts-framework repository at commit e7c7c65.

In scope were the following files:

```
packages
└── contracts
    └── src
        ├── modules
        │   ├── ERC7579MultisigConfidential.sol
        │   ├── ERC7579MultisigWeightedConfidential.sol
        │   └── validators
        │       ├── ERC7579MultisigConfidentialValidator.sol
        │       └── ERC7579MultisigWeightedConfidentialValidator.sol
        └── utils
            └── MultisigCoordinator.sol
```

Update: *All resolutions and the final state of the audited codebase mentioned in this report are contained at commit f70475d.*

System Overview

This audit covers the confidential counterparts of the ERC-7579 multisignature and validator modules (`ERC7579Multisig`, `ERC7579MultisigWeighted`, `ERC7579Validator`) audited in a first phase, along with a new utility contract that bridges multisig accounts with Zama's Fully Homomorphic Encryption (FHE) infrastructure.

The architecture follows the same ERC-7579 modular design principles established in the first phase: accounts install external modules for specific behaviors while preserving interoperability across implementations. As before, the effective security properties depend not only on each module's correctness but also on the account's module permissioning, the installation/uninstallation lifecycle, and the authorization policies implemented in derived modules.

The core of the scope is `ERC7579MultisigConfidential`, an abstract module that mirrors the non-confidential `ERC7579Multisig` but replaces plaintext storage with FHEVM encrypted types: signer identities are stored as encrypted `euint256` handles, the approval threshold as an encrypted `euint32`, and signature validation is performed homomorphically — the contract computes an encrypted boolean indicating whether the quorum was reached without revealing the threshold or which encrypted handles correspond to which signers. Note that the plaintext signer identities and their signatures are revealed during validation. It supports both address-based signers (EOAs and ERC-1271 contracts, cast to `uint256`) and ERC-7913 signers (hashed with `keccak256(verifier || key)`). Because the validation result is encrypted, a two-step disclosure pattern is required: `requestDiscloseOperation` validates the multisig and makes the encrypted result publicly decryptable (triggering asynchronous decryption by the key management service KMS), then `discloseOperation` verifies the decryption proof and sets a public `disclosed` flag.

`ERC7579MultisigWeightedConfidential` extends this base with per-signer encrypted weights (`euint16`), enabling governance schemes where some signers carry more weight than others. Each signer has a base weight of 1 plus an encrypted extra weight, and the threshold is capped at total weight instead of signer count. The maximum signer count is restricted to `type(uint16).max` (65535) to prevent overflow in total weight calculations.

Two concrete validator modules integrate the confidential multisig with EIP-4337 `UserOperation` validation. `ERC7579MultisigConfidentialValidator` implements an inline FHE decryption proof flow: when the user operation signature includes a relayer address (first 20 bytes) followed by a valid decryption proof, the validator accepts the operation without

requiring a prior call to `discloseOperation`, reducing the disclosure flow to two steps. It falls back to checking the public `disclosed(hash)` flag if the proof is absent or invalid. `ERC7579MultisigWeightedConfidentialValidator` combines the weighted and validator functionalities.

Finally, `MultisigCoordinator` is an abstract utility that addresses two limitations of multisig accounts interacting with FHE-encrypted values. First, when a multisig account needs to use an encrypted input (e.g., a transfer amount for a confidential token), only one signer can encrypt the value off-chain — the remaining signers must trust the submitter without being able to verify the encrypted content. Second, while the smart contract account may hold ACL permissions over encrypted handles, its individual signers' EOAs cannot decrypt those values, since the FHE ACL does not propagate through the account to its signers. The coordinator solves both problems: (1) any caller can submit an externally encrypted FHE value with its input proof, and the coordinator verifies the proof and grants ACL permissions to both itself and the target account; and (2) authorized signers — verified against the account's registered non-confidential `ERC7579Multisig` module — can grant ACL permission over existing encrypted handles to their own EOAs or other addresses, enabling them to request decryption from the key management service (KMS). A single deployment can serve multiple accounts.

Privacy Guarantees

The confidential multisig modules are designed to hide the following on-chain:

- **Registered signer identities:** Stored as encrypted `euint256` handles. On-chain observers can see the number of encrypted handles in the set but cannot determine the underlying signer addresses or keys. However, the identities of **participating** signers are revealed during validation: `requestDiscloseOperation` requires plaintext signer identities and their signatures as calldata, which is publicly visible in the transaction. Only the registered set remains confidential; signers who actively sign an operation are exposed.
- **Threshold value:** Stored as an encrypted `euint32`. The required number of approvals is hidden.
- **Signer weights (weighted variant):** Stored as encrypted `euint16` extra weights. Individual signer influence is hidden.
- **Validation results:** The outcome of multisig validation (whether the quorum was reached) is an encrypted `ebool` until disclosed.

The following information is **not** hidden and remains public:

- The number of encrypted signer handles registered for an account.
- The user operation payload once submitted to the bundler mempool.
- The final `disclosed(hash)` boolean after explicit disclosure.
- All event emissions, which include encrypted handles as `bytes32` values. The handles themselves do not reveal the underlying plaintext, but they are publicly visible and can be used to track operations (e.g., signer additions/removals, threshold changes, disclosure requests).

Security Model and Trust Assumptions

Throughout the in-scope codebase, the following security assumptions were identified:

- **Account enforcement (external trust assumption):** The account correctly restricts signature validation to installed modules. All signer/threshold/weight management functions use `msg.sender` as the account identifier, so the ERC-7579 account is trusted to correctly restrict who can call `onInstall`, `addSigners`, `removeSigners`, `setThreshold`, `setSignerWeights`, and `onUninstall`.
- **FHEVM and KMS infrastructure trust:** The contracts rely on Zama's FHEVM coprocessor for homomorphic operations and on the Key Management Service (KMS) for asynchronous decryption. Both are assumed to be correct, available, and honest. The audit scope covers the Solidity contracts only — the coprocessor, KMS, and ACL contract implementations were not reviewed.
- **Threshold configuration responsibility:** Due to the nature of FHE encryption, the threshold value cannot be validated against zero at runtime. An encrypted threshold of 0 allows arbitrary execution (i.e., `FHE.ge(anyValue, 0)` is always true). Account operators must rely on external validation to ensure a non-zero threshold is configured.
- **FHE non-determinism and duplicate signers:** Encrypting the same plaintext `uint256` value multiple times produces different ciphertexts and thus different encrypted handles. This means the same underlying signer can be added multiple times with different handles, and duplicate detection by comparing encrypted handles is not possible. Adding duplicate signers is not inherently harmful — it can be used intentionally to obscure the real number of unique signers — but it may cause the threshold to become unreachable, since `_validateSignatures` deduplicates by plaintext signer ID. As

with the encrypted threshold (which cannot be validated against zero on-chain), account operators must ensure the overall configuration is valid off-chain.

- **No replay protection in multisig validation:** The multisig module does not enforce replay protection for operation hashes. Replay protection and other considerations about the operation must be handled by the application layer.
- **Permanent FHE ACL grants:** ACL permissions granted via `FHE.allow` and `Impl.allow` are permanent and irrevocable. If a signer is removed from the multisig after being granted access to encrypted handles via `MultisigCoordinator`, they retain decryption rights. To revoke a former signer's access, sensitive values must be re-encrypted using fresh FHE inputs and access granted only to current signers.
- **MultisigCoordinator is incompatible with confidential multisig modules:** The `MultisigCoordinator` verifies signer membership by calling `isSigner(address, bytes)` on the registered `ERC7579Multisig` module, which stores signers in plaintext. The confidential variants expose a different function signature (`isSigner(address, bytes32)` for encrypted handles) and cannot verify plaintext signer identities, making them fundamentally incompatible with the coordinator. Accounts that use only a confidential multisig module cannot use `allowHandle`.
- **`requestDiscloseOperation` has no access control:** Anyone can call `requestDiscloseOperation` for any account and hash. This is by design: the encrypted result reveals nothing without the corresponding decryption proof.
- **ERC-7913 verifiers must use canonical key encoding:** For a given verifier, different key-byte strings must not represent the same signer. Otherwise, one signer could be registered multiple times under different encodings and be counted more than once.
- **Signer quality is the integrator's responsibility:** The contracts do not validate that signers are controlled, non-recursive, or represent appropriate entities. Adding the account's own address as a signer can create recursive ERC-1271 validation loops. Adding `address(0x04)` (identity precompile) allows arbitrary validation.
- **Singleton module deployment:** The ERC-7579 modules are designed as singletons — a single deployed contract instance serves multiple accounts. Per-account state is scoped using `msg.sender` as the account identifier (e.g., `_signersSetByAccount[account]`, `_thresholdByAccount[account]`). Storage that is not properly scoped by account can create cross-account interference.
- There are no hardcoded admin or owner roles in the base contracts. All privileged actions flow through the account or are gated by abstract validation functions that must be implemented by derived modules.

Critical Severity

C-01 Global Disclosed Mapping Shared Across Accounts Enables Cross-Account Validation Bypass

The `_disclosed` mapping is keyed only by `hash`, not by account. In contrast, signer sets and thresholds are properly scoped per account. Similarly, `_requestedDisclosure` is scoped by relayer but not by account. The `_rawERC7579Validation` function does not use the `account` parameter at all, meaning it looks up disclosure state without distinguishing which account the disclosure was validated for.

Since the validator module is a singleton shared by multiple accounts, an attacker can exploit the unscoped `_disclosed` mapping to take over any victim account on the same module instance. The module does not verify that `msg.sender` is an ERC-7579 compliant account, so any contract or EOA can interact with it directly. The attack proceeds as follows:

1. The attacker calls `onInstall` on the shared module to register attacker-controlled encrypted signers under its own address.
2. The attacker computes the victim's `userOpHash` H off-chain. This hash is deterministic from public inputs (victim address, nonce from the EntryPoint, attacker-chosen calldata, chain ID). The attacker's signers then sign H.
3. The attacker calls `requestDiscloseOperation` specifying its own address as the `account` and H as the `hash`. This function is permissionless and accepts any **arbitrary hash**. It validates signatures against the caller-specified account's signer set (the attacker's), not the account the hash belongs to. Validation passes because the attacker controls the signers. The encrypted result is marked as publicly decryptable.
4. After the KMS produces the decryption proof, the attacker calls `discloseOperation`. This sets `_disclosed[H] = true` in the global mapping, not scoped to the attacker's address.
5. The attacker submits a UserOperation for the victim with an empty signature. The EntryPoint computes the same hash H (identical inputs) and calls the validator, which falls back to `disclosed(H)` and returns success. The EntryPoint executes the attacker's calldata on the victim's account.

Consider scoping `_disclosed` by account and updating `_rawERC7579Validation` to use the `account` parameter when looking up disclosure state.

Update: *Resolved in pull request #71. The OpenZeppelin development team stated:*

> *The `disclosed` mapping is now tied to each account to avoid global disclosure.*

High Severity

H-01 `allowHandle` Does not Bind `handle` to `account`, Enabling Cross-account Permanent Decryption Grants

`MultisigCoordinator` is intended to (a) verify external FHE inputs and register ACL permissions, and (b) let authorized multisig signers extend ACL access to additional recipients. When importing a handle, `_fromExternal` always grants the coordinator itself ACL permission on the resulting `handle` via `Impl.allow(handle, address(this))`, so that subsequent calls can re-share that handle.

However, `allowHandle` accepts an arbitrary caller-supplied `handle` and only checks signer authorization against the provided `account` via `_validateAccess`. No check enforces that the `handle` was created for `account`, that `account` is already ACL-allowed on the `handle`, or that the coordinator's ACL permission originated from `account`'s flow. Since handles are `bytes32` values that are commonly observable in calldata/events/storage, any signer of any registered multisig can invoke `allowHandle` to grant themselves (or any address) decryption access to any handle the coordinator is allowed on, breaking confidentiality across accounts. The impact is amplified by the documented permanence and irreversibility of the ACL grant in `allowHandle`.

An example attack would be a victim importing a ciphertext via `fromExternal`. The coordinator and `account` become allowed to the returned `handle` via `_fromExternal` calling `Impl.allow(handle, address(this))` and `Impl.allow(handle, account)`. Attacker registers a multisig module for an attacker-controlled account via `setERC7579Multisig` and becomes a valid signer. Then he calls `allowHandle(attackerAccount, handle, attackerEOA)` using the victim's `handle` and the coordinator executes `Impl.allow(handle, attackerEOA)`, which permanently grants decryption rights to the attacker.

Consider binding imported handles to an account and enforcing that relationship in `allowHandle` (for example, store `handle => account` during `_fromExternal` and require that `account` matches, or require `Impl.isAllowed(handle, account)` before re-sharing). Consider also scoping coordinator permissions per account (e.g., per-account coordinator instances or per-account handle registries) to reduce the impact scope of a coordinator-wide ACL.

Update: *Resolved in pull request #70. The OpenZeppelin development team stated:*

> *The `allowHandle` function now checks that the `account` argument is allowed to use the `handle` argument.*

H-02 Publicly Decryptable `valid` Bit Acts as Oracle Over Secret Signer Set/Threshold/Weights

`ERC7579MultisigConfidential` aims to keep signer identities and thresholds confidential by storing encrypted signer IDs and comparing them inside `_validateMultisig`. The signer handle set is explicitly enumerable via `getSigners`, and signer handle values are also exposed in events emitted by `_addSigners` and `_removeSigners`.

However, `requestDiscloseOperation` is permissionless and lets any caller choose `signingSigners`, `signerIdHandles`, and `signatures` that are fed into `_validateSignatures`. Inside that loop, plaintext-derived signer IDs are compared against attacker-selected ciphertext handles via `FHE.eq(...)`, and the resulting threshold comparison determines the encrypted `valid` result. That `valid` bit is then made publicly decryptable in `_requestDiscloseOperation`. When genuine signatures for a given `hash` are observable and reusable, the publicly decryptable `valid` value becomes a 1-bit oracle over the confidential threshold and weight configuration. By querying with different signer subsets, it becomes possible to infer the exact threshold via binary search; in the weighted variant, analogous queries can also leak weight distribution via `ERC7579MultisigWeightedConfidential._signerWeight`. The oracle cannot reveal non-participating signers, since `_validateSignatures` requires both a valid `FHE.eq` match and a valid signature for weight to count, and the signer-to-handle mapping for participating signers is already visible in calldata. The fundamental weakness underlying the vulnerability is the fact that a permissionless caller can cause the contract to produce a secret-dependent encrypted boolean and make it publicly decryptable.

Consider removing public enumeration of signer handles (or restricting it to the account), restricting `requestDiscloseOperation` to authorized callers, and avoiding `FHE.makePubliclyDecryptable(valid)` for a predicate that depends on confidential configuration. Consider alternative designs where disclosure proofs are only usable by the account or relayer, or where on-chain results do not provide an attacker-controlled oracle over the encrypted signer set and threshold.

Update: *Partially Resolved in pull request #75. The OpenZeppelin development team stated:*

> *We concluded viable access control mitigations do not exist: restricting to the account is structurally circular in ERC-4337, and restricting to a pre-authorized relayer would require injecting opinionated access control into a singleton module. The oracle is inherently bounded to parties who hold valid cryptographic signatures. We have documented this as a design boundary in the contract NatSpec, along with a signer-reset pattern as the recommended mitigation.*

Medium Severity

M-01 Stored Disclosure Snapshots Remain Usable After Signer and Threshold Changes

The confidential multisig flow separates multisig validation from later execution. The on-chain call to `requestDiscloseOperation` computes an encrypted boolean `valid` and persists it in `_requestedDisclosure[relayer][hash]` after calling `FHE.makePubliclyDecryptable(valid)`. In the validator-based flow, `_rawERC7579Validation` later accepts if a provided proof shows that the stored disclosure result decrypts to `true`.

However, disclosure results are snapshots: neither `_requestedDisclosure` nor `_disclosed` are scoped to a signer-set or threshold version, and configuration changes do not invalidate outstanding entries. Functions that mutate configuration (`addSigners`, `removeSigners`, `setThreshold`) and even `onUninstall` do not clear or version-bump disclosures. This creates a stale-approval window that does not exist in the synchronous (non-confidential) multisig variant, where signatures are re-validated against the current config at

execution time. An operation validated under a 2-of-3 config remains executable after the config is changed to 3-of-5, as long as the nonce has not been consumed.

1. Under the old configuration (2-of-3 with signers A, B, C), an attacker gathers signatures that satisfy the threshold for hash H.
2. The attacker calls `requestDiscloseOperation`, storing a `valid = true` snapshot in `_requestedDisclosure`.
3. The account removes signer A or raises the threshold via `removeSigners` or `setThreshold`.
4. The attacker submits the operation with an inline proof, and `_rawERC7579Validation` accepts based on the stored snapshot.

While there are feasible fixes that would mitigate the above attack (see recommendations at the end), there is a corner case that would still bypass them leaving the system vulnerable. Namely, this is the presence of revocable external signers e.g., ERC-1271 contracts in the co-signer set. Such signers can change state externally and this change is not trivial to be detected by the multisigner contract.

Consider for example the following scenario:

1. An ERC-1271 signer for the account has signed a given operation hash as part of a threshold-satisfying signer set.
2. requestDiscloseOperation is called for that hash.
3. This flows through _validateMultisig, then _validateSignatures, which checks each collected signature via SignatureChecker.isValidSignatureNow.
4. If the validation result reaches threshold, it is passed through _validateThreshold and snapshotted by _requestDiscloseOperation as publicly decryptable disclosure state.
5. Later, the ERC-1271 signer contract changes state and would now reject the same signature if checked live.
6. A later validation reaches isValidSignatureWithSender, then _rawERC7579Validation.
7. At that point the validator reuses the previously stored disclosure result instead of re-checking the signature against the signer contract. So the account can still accept authorization even though a live ERC-1271 check would now fail.

Consider introducing a per-account configuration version counter (incremented on any signer, threshold, or weight change and on uninstall) and binding disclosure state to that version. Consider also proactively clearing stored disclosures on configuration changes and adding a time-based expiry to disclosure snapshots to limit long-lived authorizations. To address the corner case, consider enforcing nonce consumption also in the cases when the state of an externally revocable signer (e.g., ERC-1271) changes (without affecting signers, threshold or

weights). If this is not feasible to implement (e.g., due to the need to perform off-chain monitoring), at minimum consider explicitly documenting that, when changes occur in an ERC-1271 signer that revoke previously valid signatures, a manual update in the multisig must be triggered so that the configuration version counter is incremented and existing disclosures tied to the previous version are invalidated

Update: *Resolved in pull request #74 and pull request #76. The OpenZeppelin development team stated:*

> *The `_requestedDisclosure` and `_disclosed` storage variables are now scoped by a per-account configuration nonce (via `Nonces`) that is incremented on every `addSigners`, `removeSigners`, and `setThreshold` call. Any disclosure snapshot obtained under a previous configuration is automatically unreachable under the new nonce. Additionally, a `dismissDisclosure` function was introduced. It allows explicitly invalidating outstanding disclosures by incrementing this same nonce. This addresses the subtle ERC-1271 staleness issue.*

Low Severity

L-01 `allowHandle` Reverts for ERC-5267 Domains That Signal Extensions (Fields Bit `0x20`)

`allowHandle` derives the EIP-712 domain from the target `account` via ERC-5267. It reads the tuple from `IERC5267(account).eip712Domain()` and reconstructs the domain separator and final digest with `fields.toDomainSeparator(...).toTypedDataHash(...)`.

OpenZeppelin's `MessageHashUtils` treats the ERC-5267 extensions flag (bit `0x20`) as unsupported and reverts instead of producing a domain separator. Since `allowHandle` always calls `toDomainSeparator` before reaching `_validateAccess`, any account that sets this bit cannot use `allowHandle` at all. This is an availability failure for the coordinator's primary feature (signer-authorized ACL delegation) for affected accounts, regardless of whether the call would have used the signature path or the empty-`signer` path.

Consider explicitly rejecting extension-bearing domains up-front with a clear error (e.g., by checking `fields` before calling `toDomainSeparator`) and documenting the limitation. Consider also adding a compatibility path for such accounts, for example by accepting a

precomputed domain separator (or digest) as an input, or by using an account-provided `domainSeparator()` mechanism when available.

Update: *Resolved in pull request #73. The OpenZeppelin development team stated:*

> *The case where `fields` sets the extensions flag is now documented in the requirements section of `allowHandle`.*

Notes & Additional Information

N-01 `_discloseOperation` is Externally Reachable, Weakening Wrapper-based Extension Safety

`ERC7579MultisigConfidential._discloseOperation` is declared `public` even though it behaves like an `internal` helper for the two public disclosure entry points, `discloseOperation(bytes32,bytes,bytes)` and `discloseOperation(address,bytes32,bytes,bytes)`. This creates a third externally callable entry point into the disclosure logic.

The noted redundancy is not directly exploitable in the current implementation because the two public wrappers do not add any checks before delegating to `_discloseOperation`. However, the visibility choice is fragile for derived contracts: if an extending contract overrides the public wrapper functions to add access control, rate limiting, accounting, or other preconditions, callers could still bypass those wrapper-level checks by invoking `_discloseOperation` directly.

Consider changing `_discloseOperation` to `internal` so the contract's extension surface matches the intended wrapper pattern and cannot be bypassed accidentally by inheriting contracts.

Update: *Resolved in pull request #72. The OpenZeppelin development team stated:*

> *The `_discloseOperation` function is now `internal virtual` instead of `public`.*

Conclusion

The reviewed files implement the confidential ERC-7579 multisig flow, its weighted variant, the corresponding validator paths, and coordinator-assisted ACL sharing for encrypted handles. Together, they introduce encrypted signer identities, encrypted threshold and weight checks, disclosure and validation workflows around publicly verifiable decryption, and a coordinator abstraction intended to let authorized multisig participants extend access to existing ciphertext handles.

Overall, the codebase is thoughtfully structured and technically sound. The implementation adheres to the fhEVM model, much of the core logic is internally coherent, with good modularity and reasonable inline documentation. The audit identified one Critical-severity issue, two High-severity issues, and two Medium-severity issue, among other lower-severity findings.

The Critical issue stems from the `_disclosed` mapping being global rather than scoped per account, allowing an attacker to disclose an arbitrary hash through their own signer set and have it accepted as valid for any other account sharing the same validator instance. One High-severity issue exploits the observation that `allowHandle` does not bind a handle to the account used for authorization, allowing a signer of one account to leverage coordinator-held ACL rights and permanently grant decryption access to a handle originating from another account's flow. The other High-severity issue leverages the fact that the publicly decryptable validation result produced during multisig verification can be used as an oracle to infer confidential threshold and weight information through adaptive probing. The Medium issue reflects that disclosure snapshots remain usable after signer or threshold changes because configuration mutations do not invalidate outstanding disclosures.

We thank the OpenZeppelin development team, and Ernesto in particular, for their active engagement throughout the audit process and for answering all our technical questions and concerns in detail.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.